-

Match 4, 2026

Foundations of Python Programming

Assignment 06

Working with Classes and Static Methods

Introduction

In this assignment, I updated the Assignment 05 script to use functions, classes, and the separation of concerns programming pattern. The goal was to organize the course registration program by moving all file processing logic into a class called FileProcessor and all input and output logic into a class called IO. Each class contains static methods that receive their data through parameters and return results, so no function reads from or writes to global variables directly.

Creating the Python Program

For this assignment, I am continuing to use PyCharm, the Integrated Development Environment created by JetBrains. The script written in this document follows the same template used in previous assignments:

1. Script header
2. Setup code
3. Class and Function Definitions
4. Main body

1. Script Header

Like every Python script in this course, the file begins with a script header (Figure 1.1) that includes the title, a short description of the program, and a change log.

```
# ------------------------------------------------------------
# Title: Assignment06
# Desc: This assignment demonstrates using functions
# with structured error handling
# Change Log: (Who, When, What)
#   RRoot,1/1/2026,Created Script
#   -,3/4/2026,Updated script for assignment 6.
# ------------------------------------------------------------
```

Figure 1.1: Python Script Header.

2. Setup Code

The setup code section defines two constants that the rest of the program depends on: the MENU multi-line string and the FILE_NAME string (Figure 2.1). These values do not change at any point during the program, so using constants makes the script easier to maintain. If the file name ever needs to change, it only needs to be updated in one place.

This section is the same as in Assignment 05 because the storage format and file name have not changed. FILE_NAME still points to "Enrollments.json".

```
# Define the Data Constants
MENU: str = '''
---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------
'''
# Define the Data Constants
# FILE_NAME: str = "Enrollments.csv"
FILE_NAME: str = "Enrollments.json"
```

Figure 2.1: Setup code with a multi-line string block.

3. Class and Function Definitions

The program uses two classes to organize its logic (Figure 3.1). FileProcessor contains the functions that read from and write to the JSON file. IO contains the functions that handle all user input and screen output. Both classes use the @staticmethod decorator on every method, which means they do not require an instance to be called.

```
class FileProcessor:...


class IO:...
```

Figure 3.1: Define classes.

4. Main Body

The program defines two global variables and reads the JSON file into students on startup before entering the main loop (Figure 4.1)

```
# Extract the data from the file
students = FileProcessor.read_data_from_file(FILE_NAME)
```

Figure 4.1: Startup file read.

The while True loop drives the program. It displays the menu, captures a validated choice, and routes to one of four actions. Figure 4.2 shows the main loop structure.

```python
# Present and Process the data
while (True):

    # Present the menu of choices
    IO.output_menu(MENU)
    menu_choice = IO.input_menu_choice()

    # Input user data
    if menu_choice == "1":  # This will not work if it is an integer!
        students = IO.input_student_data(students)
        continue

    # Present the current data
    elif menu_choice == "2":
        IO.output_student_courses(students)
        continue

    # Save the data to a file
    elif menu_choice == "3":
        FileProcessor.write_data_to_file(FILE_NAME, students)
        continue

    # Stop the loop
    elif menu_choice == "4":
        break  # out of the loop

print("Program Ended")
```

Figure 4.2: Main loop structure.

Notice that menu choice 1 assigns the return value of input_student_data() back to students. This is how new records are added without the function touching the global variable directly. Figure 4.3 demonstrates how this refactoring would show up in the console.

```
What would you like to do: 1
Enter the student's first name: john
Enter the student's last name: doe
Please enter the name of the course: Py200
You have registered john doe for Py200.


---- Course Registration Program ----
  Select from the following menu:
    1. Register a Student for a Course
    2. Show current data
    3. Save data to a file
    4. Exit the program
----------------------------------------



What would you like to do: 3
The following data was saved to file!
Student Bob Smith is enrolled in Python 100
Student Sue Jones is enrolled in Python 100
Student john doe is enrolled in py2000
Student john doe is enrolled in Py200
```

Figure 4.3: Adding a student and saving to file (menu choices 1 then 3).

Summary

In this assignment, I updated the Assignment 05 script to use functions, classes, and the separation of concerns pattern. The program now organizes all file operations inside the FileProcessor class and all input and output operations inside the IO class. Every function uses the @staticmethod decorator, receives its data through parameters, and returns results instead of reading or writing global variables. Structured error handling with try/except/finally blocks is used in all functions that perform file operations or collect

user input, and all error output is routed through the shared output_error_messages function.

Github repository: https://github.com/karluw26/IntroToProg-Python-Mod06/ (external link)